

# OE-EC604B — Operating System

Semester VI • 3L:0T:0P • 3 Credits

## Module 5 Exam Notes: Deadlocks

CGEC | B.Tech ECE (2023–27) | *Biraj Sarkar*

---

### Contents

---

|          |                                                   |          |
|----------|---------------------------------------------------|----------|
| <b>1</b> | <b>System Model and Deadlock Characterization</b> | <b>2</b> |
| 1.1      | Four Necessary Conditions (Coffman Conditions)    | 2        |
| 1.2      | Resource Allocation Graph (RAG)                   | 2        |
| <b>2</b> | <b>Deadlock Handling Strategies</b>               | <b>2</b> |
| 2.1      | 1. Deadlock Prevention                            | 2        |
| 2.2      | 2. Deadlock Avoidance                             | 3        |
|          | 2.2.1 Banker's Algorithm                          | 3        |
| 2.3      | 3. Deadlock Detection & Recovery                  | 4        |

## 1. System Model and Deadlock Characterization

---

A **deadlock** is a state where a set of processes are blocked because each process is holding a resource and waiting for another resource held by some other process in the set.

### 1.1 Four Necessary Conditions (Coffman Conditions)

A deadlock situation can arise if and only if all four of the following conditions hold simultaneously in a system: 1. **Mutual Exclusion:** At least one resource must be held in a non-shareable mode (only one process can use it at a time). 2. **Hold and Wait:** A process must be holding at least one resource and waiting to acquire additional resources that are currently being held by other processes. 3. **No Preemption:** Resources cannot be preempted; a resource can be released only voluntarily by the process holding it, after that process has finished its task. 4. **Circular Wait:** A closed loop of processes must exist, where each process waits for a resource held by the next process in the loop. E.g.,  $P_0$  waits for  $R_1$  (held by  $P_1$ ),  $P_1$  waits for  $R_2$  (held by  $P_2$ ), and  $P_n$  waits for  $R_0$  (held by  $P_0$ ).

### 1.2 Resource Allocation Graph (RAG)

Deadlocks can be described precisely using a directed graph called a **Resource Allocation Graph (RAG)**:

- **Vertices ( $V$ ):** Divided into two partitions:
  - Processes  $P = \{P_1, P_2, \dots, P_n\}$  (represented by circles).
  - Resource types  $R = \{R_1, R_2, \dots, R_m\}$  (represented by rectangles containing dots for instances).
- **Edges ( $E$ ):**
  - **Request Edge:** Directed edge  $P_i \rightarrow R_j$  (process  $P_i$  has requested resource  $R_j$ ).
  - **Assignment Edge:** Directed edge  $R_j \rightarrow P_i$  (resource instance in  $R_j$  is allocated to process  $P_i$ ).
- **Theorem:**
  - If the RAG contains no cycles  $\rightarrow$  No deadlock.
  - If the RAG contains a cycle  $\rightarrow$
  - If single instance per resource type: Deadlock exists (cycle is necessary and sufficient).
  - If multiple instances per resource type: Deadlock \*may\* exist (cycle is necessary but not sufficient).

## 2. Deadlock Handling Strategies

---

Operating systems use three main strategies to handle deadlocks:

### 2.1 1. Deadlock Prevention

Ensures that the system can never enter a deadlock state by designing the system such that at least one of the four Coffman conditions cannot hold:

- **Prevent Mutual Exclusion:** Make resources shareable (e.g., read-only files). \*Drawback:\* Some resources (printers, mutexes) are inherently non-shareable.
- **Prevent Hold and Wait:** Require processes to request all resources at once before execution, or release current resources before requesting new ones. \*Drawback:\* Low resource utilization; starvation.
- **Prevent No Preemption:** If a process holding resources requests another that cannot be immediately allocated, all its current resources are preempted (released). \*Drawback:\* Overhead of saving/restoring process states.
- **Prevent Circular Wait:** Impose a global ordering on all resource types (e.g.,  $F(R_j) = i$ ) and require processes to request resources only in strictly increasing order of enumeration. \*Advantage:\* Most practical prevention strategy.

## 2.2 2. Deadlock Avoidance

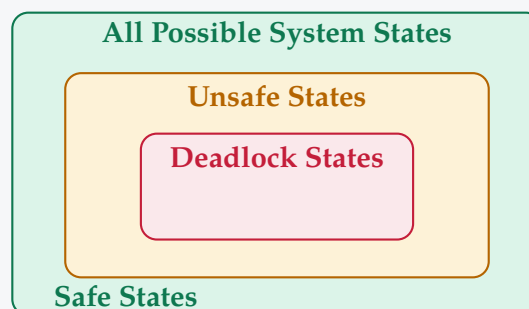
Requires that the OS has prior knowledge of the maximum resources a process will ever request.

### Safe State vs Unsafe State

A state is **safe** if there exists a **safe sequence**  $\langle P_1, P_2, \dots, P_n \rangle$  of execution such that, for each  $P_i$ , the resources that  $P_i$  can still request can be satisfied by the currently available resources plus the resources held by all prior processes  $P_j$  (for  $j < i$ ).

- **Relationship:** A safe state is not deadlocked. An unsafe state is not necessarily deadlocked, but it can lead to a deadlock if processes request their maximum limits. Avoidance algorithms keep the system inside a safe state.

### Deadlock State Space Partitioning



### 2.2.1 Banker's Algorithm

Used for systems with multiple instances of resources. Data structures:

- `Available[m]`: Available instances of each resource.
- `Max[n][m]`: Maximum demand of each process.
- `Allocation[n][m]`: Resources currently allocated to each process.
- `Need[n][m]`: Remaining resources a process may request. Note:  $\text{Need}[i][j] = \text{Max}[i][j] - \text{Allocation}[i][j]$ .

### 2.3 3. Deadlock Detection & Recovery

If a system does not prevent or avoid deadlocks, a deadlock may occur. The OS must provide:

- A detection algorithm (periodically runs to find cycles or execute safety checks).
- A recovery scheme:
  - **Process Termination:** Abort all deadlocked processes, or abort one-by-one until the cycle is broken.
  - **Resource Preemption:** Preempt resources from some processes, roll back those processes to a safe checkpoint, and restart them.

## Quick Revision — Module 5

Deadlocks | OE-EC604B

### Banker's Safety Algorithm Equations

Let  $Work = Available$  and  $Finish[i] = false$  for  $i = 0, 1, \dots, n - 1$ . Find an index  $i$  such that both: 1.  $Finish[i] == false$  2.  $Need_i \leq Work$  If such an  $i$  exists, update:

$$Work = Work + Allocation_i$$

$$Finish[i] = true$$

Repeat. If  $Finish[i] == true$  for all  $i$ , the system is in a **Safe State**.

### One-Line Definitions

- **Deadlock:** State where processes wait indefinitely for resources held by each other.
- **Mutual Exclusion:** Rule that only one process can hold a resource instance at any time.
- **Hold and Wait:** Process holding resources while requesting more without releasing current ones.
- **Preemption:** Forcible retrieval of allocated resources from a process by the OS.
- **Circular Wait:** Chain of processes where each waits for a resource held by the next.
- **Resource Allocation Graph:** Directed graph showing resource allocations and process requests.
- **Safe Sequence:** Order of process execution that guarantees no deadlock can occur.
- **Ostrich Algorithm:** Deadlock strategy of ignoring the problem (used when deadlocks are rare).

### Frequently Asked / Viva Questions

**Q1. What are the four necessary conditions for a deadlock to occur?**

Mutual Exclusion, Hold and Wait, No Preemption, and Circular Wait (Coffman conditions). All four must hold simultaneously for a deadlock to exist.

**Q2. Consider a system with 5 processes (P0 to P4) and 3 resource types (A, B, C):**

$$\text{Allocation} = \begin{pmatrix} 0 & 1 & 0 \\ 2 & 0 & 0 \\ 3 & 0 & 2 \\ 2 & 1 & 1 \\ 0 & 0 & 2 \end{pmatrix}, \quad \text{Max} = \begin{pmatrix} 7 & 5 & 3 \\ 3 & 2 & 2 \\ 9 & 0 & 2 \\ 2 & 2 & 2 \\ 4 & 3 & 3 \end{pmatrix}, \quad \text{Available} = (3 \ 3 \ 2)$$

## Frequently Asked / Viva Questions (contd.)

**Is the system in a safe state? Find the safe sequence.**

\*Step 1: Calculate Need matrix (Need = Max – Allocation)\*

$$\text{Need} = \begin{pmatrix} 7 - 0 & 5 - 1 & 3 - 0 \\ 3 - 2 & 2 - 0 & 2 - 0 \\ 9 - 3 & 0 - 0 & 2 - 2 \\ 2 - 2 & 2 - 1 & 2 - 1 \\ 4 - 0 & 3 - 0 & 3 - 2 \end{pmatrix} = \begin{pmatrix} 7 & 4 & 3 \\ 1 & 2 & 2 \\ 6 & 0 & 0 \\ 0 & 1 & 1 \\ 4 & 3 & 1 \end{pmatrix}$$

\*Step 2: Initialize Work = (3 3 2)\*

- Check P1:  $\text{Need}_1 (1 \ 2 \ 2) \leq \text{Work} (3 \ 3 \ 2)$ ? Yes.  $\text{Work} \rightarrow \text{Work} + \text{Alloc}_1 = (5 \ 3 \ 2)$ .

- Check P3:  $\text{Need}_3 (0 \ 1 \ 1) \leq \text{Work} (5 \ 3 \ 2)$ ? Yes.  $\text{Work} \rightarrow \text{Work} + \text{Alloc}_3 = (7 \ 4 \ 3)$ .

- Check P0:  $\text{Need}_0 (7 \ 4 \ 3) \leq \text{Work} (7 \ 4 \ 3)$ ? Yes.  $\text{Work} \rightarrow \text{Work} + \text{Alloc}_0 = (7 \ 5 \ 3)$ .

- Check P2:  $\text{Need}_2 (6 \ 0 \ 0) \leq \text{Work} (7 \ 5 \ 3)$ ? Yes.  $\text{Work} \rightarrow \text{Work} + \text{Alloc}_2 = (10 \ 5 \ 5)$ .

- Check P4:  $\text{Need}_4 (4 \ 3 \ 1) \leq \text{Work} (10 \ 5 \ 5)$ ? Yes.  $\text{Work} \rightarrow \text{Work} + \text{Alloc}_4 = (10 \ 5 \ 7)$ .

\*Result:\* All processes finished. System is in **safe state** with safe sequence: **(P1, P3, P0, P2, P4)**.

### Q3. Differentiate between Deadlock Prevention and Deadlock Avoidance.

Prevention limits system design by enforcing policies that violate one of the 4 Coffman conditions (e.g., resource ordering). Avoidance does not restrict request structures but checks dynamic resource availability and maximum limits to keep the system in a safe state.

### Q4. How can the circular wait condition be prevented in operating systems?

By defining a global 1-to-1 enumeration function  $F : R \rightarrow \mathbb{N}$  for all resource types. A process can only request resource  $R_j$  if  $F(R_j) > F(R_i)$ , where  $R_i$  is any resource currently held by the process.

### Q5. What is an unsafe state? Does it always lead to a deadlock?

An unsafe state is a state from which the OS cannot guarantee that all processes can finish their maximum execution. It does **not** always lead to a deadlock; it will only deadlock if processes actually request their maximum declared resources.

## Frequently Asked / Viva Questions (contd.)

**Q6. Why does a cycle in a Resource Allocation Graph (RAG) not always indicate a deadlock?**

If a resource type has multiple instances, a cycle indicates a potential deadlock but not a guaranteed one, as a process outside the cycle might release an instance, breaking the cycle.

**Q7. What is the difference between a Resource Allocation Graph and a Wait-For Graph?**

A Resource Allocation Graph shows both processes and resources with request/assignment edges. A Wait-For Graph is a simplified version for single-unit resource systems, showing only processes, where edge  $P_i \rightarrow P_j$  means  $P_i$  waits for  $P_j$  to release a resource.

**Q8. Explain the recovery methods from deadlock.**

1. Process Termination: Abort all deadlocked processes, or abort processes one-by-one until the cycle is broken. 2. Resource Preemption: Take resources from a process, roll it back to a safe checkpoint, and restart it.

**Q9. What is the "Ostrich Algorithm" and when is it appropriate to use?**

It is the strategy of ignoring deadlocks altogether, assuming they occur rarely. It is used by most general-purpose OS (like Windows/Linux) because deadlock prevention/avoidance adds heavy run-time overhead.

**Q10. Why is preemption difficult to implement for some resources?**

Because some resources (like printers or files being written) cannot be easily saved and restored mid-operation without corrupting data or output.

## Deadlock Handling Trade-offs

| Strategy   | Run-time Overhead        | Resource Utilization   | Implementation             |
|------------|--------------------------|------------------------|----------------------------|
| Prevention | None (static rules)      | Low (strict limits)    | Simple                     |
| Avoidance  | Moderate (safety checks) | Moderate               | Complex (needs max demand) |
| Detection  | High (periodic checks)   | High (free allocation) | Moderate                   |